AGRICULTURE MANAGEMENT SYSTEM

A SUMMER INTERNSHIP REPORT

Submitted by

SANGEETHA M– 810023205105



.

DEPARTMENT OF INFORMATION TECHNOLOGY

ANNA UNIVERSITY ,

TIRUCHIRAPALLI – 620 024.

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS),

ANNA UNIVERSITY , TIRUCHIRAPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY



BONAFIDE CERTIFICATE

This is to certify this Summer Internship project report **"Agriculture Management System"** is the Bonafide work of **SANGEETHA M (810023205105)** has satisfactory completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2025-2026.**

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on _____.

ABSTRACT

The Agricultural Management System is a full-stack web application developed to modernize and simplify agricultural activities. The system provides a centralized digital platform where farmers can access essential agricultural information, manage crop-related details, check weather conditions, receive irrigation guidance, view market prices, and explore various government schemes. It helps farmers make informed decisions and improves the overall efficiency of agricultural management.

The project is implemented using Java with Spring Boot for the backend, providing a scalable architecture for handling business logic and RESTful web services. The frontend is developed using HTML, CSS, JavaScript, and Bootstrap, providing a responsive and user-friendly interface. The integration of these technologies enables efficient communication between the frontend and backend.

One of the main objectives of the system is to reduce the difficulties faced by farmers in accessing agricultural information, improve crop management, provide irrigation guidance, deliver weather-related information, display market prices, and provide government scheme information. By bringing these services together on a single platform, the system helps farmers make better agricultural decisions.

Overall, the Agricultural Management System demonstrates the practical application of full-stack development and modern software engineering principles in the agricultural domain. It combines a Java-based backend with an interactive web interface to support farmers, improve decision-making, optimize resource utilization, and enhance the overall efficiency of agricultural management.

prices, and explore various government schemes. It helps farmers make informed decisions and improves the overall efficiency of agricultural management.

The project is implemented using **Java with Spring Boot** for the backend, providing a scalable architecture for handling business logic and RESTful web services. The frontend is developed using **HTML, CSS, JavaScript, and Bootstrap**, providing a responsive and user-friendly interface. The integration of these technologies enables efficient communication between the frontend and backend.

One of the main objectives of the system is to reduce the difficulties faced by farmers in accessing agricultural information, improve crop management, provide irrigation guidance, deliver weather-related information, display market prices, and provide government scheme information. By bringing these services together on a single platform, the system helps farmers make better agricultural decisions.

Overall, the **Agricultural Management System** demonstrates the practical application of full-stack development and modern software engineering principles in the agricultural domain. It combines a Java-based backend with an interactive web interface to support farmers, improve decision-making, optimize resource utilization, and enhance the overall efficiency of agricultural management.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
1	INTRODUCTION 1.1 OBJECTIVE OF THE INTERNSHIP 1.2 MOTIVATION	1 – 9
2	COMPANY PROFILE	10 – 12
3	DAILY WORK REPORT	13 – 16
4	BLOCK DIAGRAM	17
5	AGRICULTURAL MANAGEMENT SYSTEM 5.1 EXISTING SYSTEM 5.2 PROPOSED SYSTEM 5.3 CAPABILITIES	17 – 20
6	PROPOSED METHOD 6.1 USER AUTHENTICATION 6.2 CROP MANAGEMENT 6.3 WEATHER MANAGEMENT 6.4 IRRIGATION GUIDANCE 6.5 MARKET PRICE MANAGEMENT 6.6 GOVERNMENT SCHEME MANAGEMENT 6.7 DATABASE MANAGEMENT 6.8 NOTIFICATION AND INFORMATION MANAGEMENT	21 – 25
7	ALGORITHM 7.1 USER LOGIN 7.2 USER REGISTRATION 7.3 CROP MANAGEMENT 7.4 WEATHER MANAGEMENT 7.5 IRRIGATION GUIDANCE 7.6 MARKET PRICE MANAGEMENT 7.7 GOVERNMENT SCHEME MANAGEMENT 7.8 ADMIN OPERATIONS	25 – 28

CHAPTER	TITLE	PAGE NO
8	CODE IMPLEMENTATION	28 – 34
9	SAMPLE OUTPUT	35 – 41
10	ANALYSIS	42 – 43
11	CONCLUSION	43 – 44
12	FUTURE WORK	44 – 45
13	REFERENCES	45 – 46

1.INTRODUCTION

The **Agricultural Management System** is a Java Full Stack web application developed to automate and simplify the management of various agricultural activities and provide essential information to farmers through a centralized digital platform.

In today's agricultural environment, farmers face several challenges in accessing timely and reliable information related to crop management, weather conditions, irrigation, market prices, and government schemes. Traditional methods of obtaining agricultural information often depend on manual sources, local communication, and scattered information, which can result in delays, inaccurate decisions, and inefficient resource utilization. To overcome these challenges, a centralized web-based **Agricultural Management System** has been developed.

The primary purpose of this project is to provide a secure and user-friendly platform where farmers can access crop-related information, check weather conditions, receive suitable irrigation guidance, view agricultural market prices, and explore various government schemes. By bringing these important agricultural services together on a single platform, the system reduces the difficulty of accessing information and supports farmers in making better and timely agricultural decisions.

The application is developed using **Java Full Stack technologies**. The frontend is designed using **HTML5, CSS3, JavaScript, and Bootstrap**, which provide a responsive and interactive user interface. The backend is developed using **Java Spring Boot**, which handles business logic, REST API development, agricultural information management, and communication between the frontend and database. **MySQL** is used as the relational database management system to store relevant agricultural data, user information, crop details, and other system-related information. **Apache Maven** is used for dependency management and project building, while **Eclipse/IntelliJ IDEA** can be used as the development environment.

The system follows a **three-tier architecture**, consisting of the Presentation Layer (Frontend), Business Logic Layer (Spring Boot Backend), and Database Layer (MySQL). The frontend communicates with the backend through RESTful APIs, enabling efficient data exchange and smooth interaction between users and the application. This layered architecture improves the maintainability, scalability, reliability, and security of the system.

The **Agricultural Management System** consists of several important modules, including **Crop Management, Weather Management, Irrigation Guidance, Market Prices, and Government Schemes**. Each module performs specific functions and works together to provide a complete agricultural information

management solution. These modules help farmers access necessary information and make appropriate decisions based on crop requirements, weather conditions, irrigation needs, market information, and available government schemes.

Overall, the **Agricultural Management System** is a practical implementation of Java Full Stack Development that demonstrates the integration of frontend technologies, Spring Boot backend services, and MySQL database management. The project provides practical knowledge in web application development, database connectivity, REST API implementation, and software engineering principles. It serves as an efficient and user-friendly solution for modern agricultural management System **.**

1.1 OBJECTIVES OF THE INTERNSHIP

Specific Objectives

The main objective of the **Agricultural Management System** is to develop a secure, user-friendly, and efficient web application using **Java Full Stack** technologies. The system is designed to simplify agricultural information management by providing a centralized platform for farmers to access crop, weather, irrigation, market price, and government scheme information effectively.

The objectives of the project are:

• To develop a web-based application using **Java Spring Boot, HTML, CSS, JavaScript, Bootstrap, and MySQL**.

• To provide secure **user registration and login** functionality.

• To enable farmers to access and manage **crop-related information** conveniently.

• To provide users with accurate and timely **weather information** for agricultural activities.

• To provide suitable **irrigation guidance** based on crop and environmental conditions.

• To provide updated information about **agricultural market prices**.

• To provide farmers with information about relevant **government agricultural schemes**.

• To reduce the difficulty of accessing agricultural information from multiple sources.

• To support farmers in making **better and timely agricultural decisions**.

• To provide a **responsive, reliable, and easy-to-use** agricultural management application.
• To improve the efficient utilization of **agricultural resources** through digital information and guidance.
• To demonstrate the practical application of **Java Full Stack development, REST APIs, database connectivity, and software engineering principles**.

1.2 MOTIVATION

The motivation behind developing the **Agricultural Management System** is to make agricultural information easily accessible, organized, and efficient for farmers. Many farmers still depend on traditional methods and multiple sources to obtain information about crops, weather conditions, irrigation, market prices, and government schemes, which can lead to delays, inaccurate decisions, and difficulty in accessing timely information.

This project provides an online platform where farmers can easily **access crop-related information, check weather conditions, receive irrigation guidance, view market prices, and explore government agricultural schemes**. It brings important agricultural services together in a single centralized system, making information access more convenient and efficient.

Another major motivation is to **reduce dependency on manual information sources, minimize difficulties in accessing agricultural information, improve resource utilization, and support better decision-making**. By providing essential agricultural information through a digital platform, the system helps farmers save time and improve the overall efficiency of agricultural activities.

This project also provides practical knowledge of **Java Full Stack Development** by using technologies such as **Spring Boot, HTML, CSS, JavaScript, Bootstrap, and**

MySQL to develop a real-world web application. It demonstrates how modern software technologies can be applied to address common agricultural challenges and support farmers effectively.

2.COMPANY PROFILE



About the Company:

Fantasy Solution is an Information Technology (IT) company that focuses on providing software development and digital technology solutions to meet the growing needs of modern businesses. The company specializes in developing web applications, software solutions, and technology-based services that help organizations improve their operational efficiency and adopt digital solutions. The primary objective of Fantasy Solution is to develop reliable, user-friendly, and efficient software applications that address real-world business requirements. The company follows modern software development practices and focuses on application design, development, database management, testing, and implementation. It also emphasizes quality, performance, maintainability, and customer satisfaction throughout the development process.

Fantasy Solution also provides industrial training and internship opportunities for students and fresh graduates. These programs help students bridge the gap between academic knowledge and practical industry requirements by providing exposure to real-time

software development projects. During the internship, students can gain practical experience in areas such as frontend development, backend development, database management, REST API development, software testing, debugging, and application deployment.

As part of my internship at Fantasy Solution, I developed a Car Parking Management System using Java Full Stack technologies. The project involved designing a responsive and user-friendly interface using HTML, CSS, JavaScript, and Bootstrap, developing backend services using Java Spring Boot, integrating the MySQL database, implementing RESTful APIs, and testing the application. The system was designed to simplify parking management by providing features such as user registration and login, parking slot management, vehicle management, parking slot booking, and administrative management.

The internship provided me with valuable practical knowledge of Java Full Stack Development and helped me understand the complete Software Development Life Cycle (SDLC). It improved my programming skills, database handling, problemsolving abilities, API development knowledge, and understanding of frontendbackend integration. Overall, the internship at Fantasy Solution provided an excellent opportunity to apply theoretical concepts in a practical software development environment. The experience helped me understand industry-oriented development practices and strengthened my technical skills in Java, Spring Boot, MySQL, HTML, CSS, JavaScript and REST API development.

Company Details:

- **Company Name:** Fantasy Solution
- **Industry:** Information Technology (IT) Services and Software Development

- **Location:** # 16,Samanth Plazza,Madurai Main Road,Melapudur,Trichy- 1,Tamil Nadu, India
- **Development Technologies:** Java, Spring Boot, HTML, CSS, JavaScript, Bootstrap, MySQL, REST APIs
- **Major Services:** Software Development, Web Application Development, Database Solutions, Mobile Application Development, Software Testing, and Technical Training
- **Internship Training:** Java Full Stack Development, Web Application Development, Database Management, REST API Development, Software Testing, and Real-Time Projects
- **Development Approach:** User-friendly, scalable, secure, reliable, and industry-oriented software solutions
- **Project Completed During Internship**: Car Parking Management System (Java Full Stack)

Services Offered

- Software Development
- Web Application Development
- Website Design and Development
- Mobile Application Development
- Java Full Stack Development
- Database Design and Management
- REST API Development
- Digital Marketing and SEO
- Domain Registration and Web Hosting
- Technical Internship and Industrial Training

Technologies Used by the Company

- **Programming Languages:** Java, JavaScript
- **Frontend:** HTML5, CSS3, JavaScript
- **Backend:** Java Spring Boot
- **Database:** MySQL
- **Development Tools:** Eclipse, IntelliJ IDEA, VS Code
- **Build Tool:** Apache Maven
- **Version Control:** Git and GitHub
- **API Technology:** RESTful Web Services

3. DAILY WORK REPORT

Day 1: Attended orientation session; briefed on internship guidelines, evaluation criteria, and reporting format. Discussed and finalized the project topic – "Travel Agency Management System" – with the faculty in-charge.

Day 2: Conducted requirement gathering by discussing with the mentor. Listed the stakeholders (Customer, Administrator) and identified the core functional requirements of the travel agency application.

Day 3: Carried out a literature review of existing travel booking platforms. Studied their features and documented the drawbacks of manual, paper-based travel agency operations.

Day 4: Finalized the scope and objectives of the project. Prepared the list of hardware and software requirements needed for development.

Day 5: Set up the development environment – installed JDK 17, IntelliJ IDEA, Apache Maven, and Git. Verified installation with a sample "Hello World" Spring Boot application.

Day 6: Studied Spring Boot fundamentals – auto-configuration, embedded Tomcat server, annotations such as @RestController, @Service, @Repository, and the standard MVC project structure.

Day 7: Designed the overall system architecture of the application using a layered approach (Presentation Layer, Business Logic Layer, and Data Layer).
Prepared the architecture diagram.

Day 8: Designed the database schema by identifying entities – User, Travel Package, Vehicle, Booking, Payment, and Support Request – along with their attributes.

Day 9: Prepared the Entity-Relationship (ER) diagram showing relationships between User, Booking, Package, Vehicle, and Payment entities.

Day 10: Created wireframes for the Home, Login, and Registration pages.
Planned the responsive layout using the Bootstrap grid system.

Day 11: Developed the Home Page using HTML, CSS, and Bootstrap, including the navigation bar, hero banner, popular package cards, and footer section.

Day 12: Developed the Login Page UI with an email/password form, "Remember Me" option, and client-side validation using JavaScript.

Day 13: Developed the Registration Page UI with fields for name, email, phone, address, role selection, and password confirmation, along with input validations.

Day 14: Initialized the Spring Boot backend project using Spring Initializr.

Organized the project into controller, service, repository, and model packages.

Day 15: Created the Model (entity) classes – User, TravelPackage, and Vehicle – defining their fields and getters/setters.

Day 16: Implemented the UserController and UserService to handle new user registration, including basic input validation and duplicate-email checks.

Day 17: Implemented the AuthController for the login module, including logic in UserService to authenticate users by email and password.

Day 18: Tested the Registration and Login REST APIs using Postman. Verified success and error responses (e.g., invalid credentials, duplicate user).

Day 19: Developed the Package Module – TravelPackage model, PackageController, and PackageService – to support adding, updating, and listing travel packages.

Day 20: Developed the Vehicle Module – Vehicle model, VehicleController, and VehicleService – to manage vehicle type, seating capacity, availability, and rental cost.

Day 21: Built the Package listing and Vehicle listing pages on the frontend and connected them to the respective backend APIs using JavaScript fetch calls.

Day 22: Developed the Booking Module – Booking model, BookingController, and BookingService – to create and store booking records for packages and vehicles.

Day 23: Designed and integrated the Booking Page on the frontend, allowing customers to select a package/vehicle, enter travel details, and submit a booking request.

Day 24: Developed the Payment Module – Payment model, PaymentController, and PaymentService – to process payments and update the transaction status.

Day 25: Designed the Payment Page frontend showing the booking summary and payment method selection, and integrated it with the Payment REST API.

Day 26: Developed the Refund Module, enabling customers to submit refund requests for cancelled bookings and allowing status tracking of each request.

Day 27: Developed the Support Module for raising and managing customer support tickets, enabling communication between customers and administrators.

Day 28: Developed the Notification Module to alert customers about booking confirmation, payment status, cancellations, and support responses.

Day 29: Developed the Admin Module – admin dashboard with statistics, and functionality to manage users, packages, vehicles, bookings, and generate reports. **Day 30:** Performed unit and integration testing of all modules, fixed identified bugs, took screenshots for documentation, and compiled the final internship report.

4. BLOCK DIAGRAM

4.1 System Block Diagram

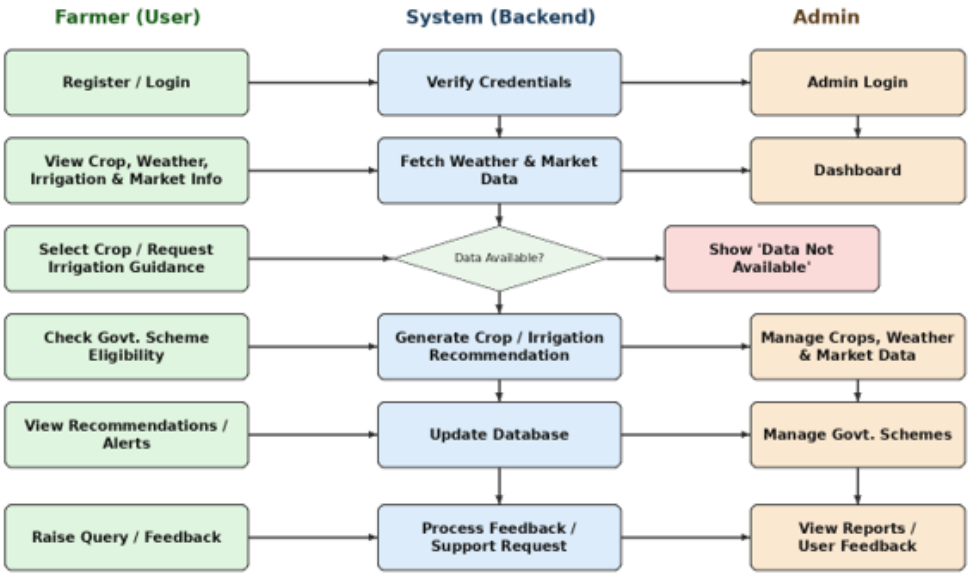


Fig 4.1: Block Diagram of the Agricultural Management System

5. AGRICULTURAL MANAGEMENT SYSTEM

5.1 Existing System

Most farmers still depend on traditional and manual methods to plan crops, monitor weather, schedule irrigation, track market prices, and learn about government schemes. Farmers typically rely on informal advice from other farmers, local agricultural officers, newspapers, or personal experience, with limited access to consolidated, up-to-date, and location-specific information.

This existing approach suffers from several limitations:

● **Manual crop planning** – Crop selection is often based on tradition or guesswork rather than season, soil, and location-specific information, increasing the risk of poor yield.

● **No real-time weather awareness** – Farmers may not have immediate access to accurate and localized weather information before planning irrigation, harvesting, or other farming activities.

● **Inefficient irrigation practices** – Without proper guidance on irrigation frequency, timing, and methods, farmers may over-irrigate or under-irrigate, leading to water wastage and affecting crop growth.

● **Fragmented market information** – Market prices for agricultural commodities are available through different sources, making it difficult for farmers to identify suitable selling opportunities.

● **Limited awareness of government schemes** – Many farmers remain unaware of beneficial government agricultural schemes due to the lack of a centralized and easily accessible information source.

● **Inefficient information management** – Managing and updating crop, weather, irrigation, market price, and government scheme information manually can be difficult without a centralized system.

● **Increased decision-making time** – Collecting and verifying information from multiple sources can delay important agricultural decisions.

● **Difficulty in maintaining agricultural records** – Maintaining crop-related and other agricultural information manually can result in missing, outdated, or inaccurate records.

These drawbacks make the existing manual approach less efficient in meeting the growing information needs of farmers. Therefore, there is a need for a digital, centralized, and farmer-friendly Agricultural Management System that can integrate crop, weather, irrigation, market price, and government scheme information, improve decision-making, and provide convenient access to essential agricultural information**.**

5.2 Proposed System

The proposed Agricultural Management System is a full-stack web application that digitizes and centralizes essential agricultural information and services. It replaces traditional methods of collecting agricultural information with an integrated online platform that can be accessed by farmers and administrators. The system is designed to simplify agricultural information management, reduce dependency on manual sources, support better decision-making, and improve the overall efficiency of agricultural activities.

The system is developed using Java Spring Boot for the backend and HTML, CSS, JavaScript, and Bootstrap for the frontend, with MySQL used for centralized data storage. Key aspects of the proposed system include:

• **Centralized platform** – Crop information, weather details, irrigation guidance, market prices, government schemes, and user information are managed through a centralized system.

• **User registration and login** – Farmers can securely register, log in, and access the agricultural services available through their accounts.

• **Crop Management** – Farmers can access crop-related information and guidance to support suitable crop planning and management.

• **Weather Management** – Users can check relevant weather information, helping them plan agricultural activities based on current and expected weather conditions.

• **Irrigation Guidance** – The system provides suitable irrigation guidance to help farmers manage water usage and meet the irrigation requirements of crops.

• **Market Prices** – Farmers can view agricultural market price information, helping them understand current price conditions and make better selling decisions.

• **Government Schemes** – The system provides information about government agricultural schemes and benefits, making it easier for farmers to identify suitable schemes.

• **Administrative management** – Administrators can manage agricultural information, update crop details, market prices, irrigation guidance, weather-related information, and government scheme details through a centralized interface.

• **Database management** – The system stores and manages user details and relevant agricultural information in a centralized MySQL database, ensuring organized and efficient data management.

• **Information and record management** – The system maintains organized agricultural information and allows users to access essential details through a simple and user-friendly interface.

By automating and centralizing these processes, the proposed Agricultural Management System reduces the difficulty of accessing agricultural information, minimizes dependency on manual sources, supports better agricultural decision-making, and provides a convenient and efficient digital platform for farmers and administrators.

5.3 Capabilities

The Agricultural Management System offers the following key capabilities.

• **User Management** – Provides secure user registration, login, authentication, and profile management for farmers and administrators.

• **Crop Management** – Enables users to access and manage crop-related information and obtain suitable guidance for crop planning and management.

• **Weather Management** – Allows farmers to check weather conditions and access weather-related information for planning agricultural activities.

• **Irrigation Guidance** – Provides suitable irrigation guidance based on crop requirements and relevant agricultural conditions to support efficient water usage.
• **Market Price Management** – Provides farmers with agricultural market price information, helping them understand price conditions and make better selling decisions.
• **Government Scheme Management** – Provides information about government agricultural schemes and benefits, helping farmers identify and access suitable schemes.
• **Information Management** – Enables administrators to add, update, and manage crop, irrigation, market price, weather, and government scheme information.
• **Database Management** – Stores user details and agricultural information securely in a centralized MySQL database.
• **Reporting and Monitoring** – Helps administrators monitor and manage agricultural information, user activities, and other system-related operations efficiently.

6. PROPOSED METHOD

6.1 User Authentication

• Handles secure registration and login for farmers and administrators to provide controlled access to the system.
• Captures user details such as name, email, phone number, password, and role during the registration process.
• Validates the entered login credentials and prevents duplicate email registrations.
• Provides role-based access, allowing farmers to access agricultural services while administrators can access the administrative dashboard.
• Protects user information by validating authentication details before granting access to the system.
• Displays appropriate error messages when invalid login credentials or incomplete registration details are provided.

6.2 Crop Management

• Manages important crop-related information such as crop name, soil type, season, location, crop requirements, and other relevant details.
• Allows administrators to add, update, and delete crop information according to the requirements of the agricultural system.
• Enables farmers to access crop information and guidance through a simple and user-friendly interface.
• Provides suitable crop-related information that can help farmers plan cultivation activities more effectively.
• Helps farmers understand crop requirements and select suitable crops based on available agricultural information.
• Maintains organized crop records in the database for efficient retrieval and future reference.

6.3 Weather Management

• Provides weather-related information to help farmers plan and schedule important agricultural activities.
• Displays available weather details such as temperature, humidity, rainfall, wind conditions, and other relevant weather information.
• Helps farmers understand current weather conditions before performing activities such as irrigation, harvesting, and crop maintenance.
• Provides timely weather information through the web application in an easily understandable format.
• Allows the system to use available weather data to support better agricultural planning and decision-making.
• Helps reduce the risks associated with unexpected weather conditions by providing relevant information to farmers.

6.4 Irrigation Guidance

• Provides suitable irrigation guidance based on crop requirements, soil conditions, weather conditions, and other relevant agricultural factors.
• Helps farmers understand the appropriate irrigation method, frequency, and timing required for different crops.
• Provides useful guidance to support efficient utilization of available water resources.
• Helps prevent unnecessary water usage caused by over-irrigation and reduces the risk of insufficient irrigation.
• Supports farmers in maintaining suitable moisture conditions required for healthy crop growth.
• Helps improve irrigation practices and contributes to better resource management in agricultural activities.

6.5 Market Price Management

• Provides agricultural market price information for different crops and agricultural commodities.
• Stores relevant market price details such as crop name, market location, price, date, and other available information.
• Enables farmers to access market price information through the web application without depending entirely on manual sources.
• Helps farmers compare available market prices and understand the current market conditions.
• Supports farmers in making better decisions regarding the suitable time and place for selling their agricultural products.

• Allows administrators to add, update, and manage market price information in the centralized system.

6.6 Government Scheme Management

• Provides information about government agricultural schemes and benefits available for farmers.
• Stores scheme-related details such as scheme name, eligibility criteria, benefits, application information, and other relevant details.
• Enables farmers to easily view and understand available government schemes through the system.
• Helps farmers identify schemes that may be suitable for their agricultural requirements.
• Reduces the difficulty of searching for scheme information from multiple sources.
• Allows administrators to add, update, and manage government scheme information whenever required.

6.7 Database Management

• Stores user, crop, weather, irrigation, market price, and government scheme information in a centralized **MySQL database**.
• Maintains organized records and relationships between users and the different agricultural information modules.
• Provides efficient data retrieval, insertion, updating, and deletion through the Spring Boot backend.
• Ensures that the information displayed to users is retrieved from the appropriate database records.
• Helps maintain data consistency, accuracy, and reliability throughout the application.
• Provides a structured approach for managing agricultural information and supports future expansion of the system.

6.8 Notification and Information Management

• Provides appropriate notifications and messages to users regarding important agricultural information and system activities.
• Displays crop, weather, irrigation, market price, and government scheme information in a clear and organized manner.
• Provides suitable messages for successful, failed, unavailable, or incomplete operations.
• Helps users understand the status of their requested operations and available information.
• Enables administrators to maintain and update important agricultural information for users.
• Improves communication between farmers and administrators and enhances the overall usability and user experience of the Agricultural Management System.

7. ALGORITHM

7.1 User Login – Algorithm

1. User enters email and password in the login form.
2. System validates whether all required fields are entered.
3. Login request is sent to the authentication controller.
4. System checks whether the entered email exists in the database.
5. System verifies the entered password with the stored password.
6. If the credentials are invalid, an appropriate error message is displayed.
7. If the credentials are valid, the user is redirected to the appropriate dashboard.

7.2 User Registration – Algorithm

1. User enters name, email, phone number, and password in the registration form.
2. System validates all required registration fields.

3. System checks whether the entered email is already registered.
4. If the email already exists, the registration request is rejected.
5. If the email is unique, a new user record is created.
6. User details are stored securely in the MySQL database.
7. On successful registration, the user is redirected to the login page.

7.3 Crop Management – Algorithm

1. User logs into the system and opens the Crop Management section.
2. System displays the available crop information.
3. User selects the required crop or enters the relevant crop details.
4. System retrieves the corresponding crop information from the database.
5. System validates whether the requested crop information is available.
6. If the information is unavailable, an appropriate message is displayed.
7. If the information is available, the relevant crop details and guidance are displayed to the user.

7.4 Weather Management – Algorithm

1. User logs into the system and opens the Weather Management section.
2. System requests the required location or weather information.
3. System retrieves the available weather data.
4. System processes the received weather information.
5. The system checks whether the requested weather information is available.
6. If the information is unavailable, an appropriate error message is displayed.

7. If available, the weather details such as temperature, humidity, rainfall, and other relevant information are displayed to the user.

7.5 Irrigation Guidance – Algorithm

1. User opens the Irrigation Guidance section.
2. User selects or enters the required crop and relevant agricultural details.
3. System validates the entered information.
4. System retrieves the corresponding irrigation guidance from the database.
5. System identifies the suitable irrigation method, timing, or frequency.
6. The system displays the recommended irrigation guidance to the user.
7. User can use the provided guidance to manage irrigation activities efficiently.

7.6 Market Price Management – Algorithm

1. User opens the Market Prices section.
2. User selects the required crop or agricultural commodity.
3. System validates the selected crop or commodity.
4. System searches the database for the corresponding market price information.
5. System retrieves the available market price details.
6. If the required information is unavailable, an appropriate message is displayed.
7. If available, the market price and related information are displayed to the user.

7.7 Government Scheme Management – Algorithm

1. User opens the Government Schemes section.

2. System retrieves the available agricultural scheme information.
3. User selects the required government scheme.
4. System searches the database for the selected scheme details.
5. System retrieves information such as scheme name, eligibility, benefits, and other available details.
6. If the scheme information is unavailable, an appropriate message is displayed.
7. If available, the complete scheme information is displayed to the user.

7.8 Admin Operations – Algorithm

1. Administrator enters valid login credentials.
2. System verifies the administrator details and opens the admin dashboard.
3. Administrator selects the required management operation.
4. System provides options to manage users, crops, weather information, irrigation guidance, market prices, and government schemes.
5. Administrator performs add, update, delete, or view operations.
6. System validates the changes and updates the MySQL database.
7. The admin dashboard displays the updated system information.

8. CODE IMPLEMENTATION

8.1 Login Module Code

Contains authentication logic for validating user credentials and providing secure access to the Agricultural Management System.

```
@RestController
@RequestMapping("/api/auth")
```

```
@CrossOrigin(origins = "http://localhost:3000")
public class AuthController {

    @Autowired
    private UserService userService;

    @PostMapping("/login")
    public ResponseEntity<?> login(
            @RequestBody LoginRequest request) {

        User user = userService.authenticateUser(
                request.getEmail(),
                request.getPassword());

        if (user != null) {
            return ResponseEntity.ok(
                    new MessageResponse("Login Successful"));
        } else {
            return ResponseEntity
                    .status(HttpStatus.UNAUTHORIZED)
                    .body(new MessageResponse(
                            "Invalid Email or Password"));
        }
    }
}
```

8.2 Registration Module Code

Handles new farmer account creation and validation.

```
@RestController
@RequestMapping("/api/users")
@CrossOrigin(origins = "http://localhost:3000")
public class UserController {
```

```
    @Autowired
    private UserService userService;

    @PostMapping("/register")
    public ResponseEntity<User> registerUser(
            @RequestBody User user) {

        User newUser = userService.registerUser(user);

        return new ResponseEntity<>(
                newUser,
                HttpStatus.CREATED);
    }
}
```

8.3 Crop Management Module Code

Manages crop information and retrieves available crop details for farmers.

```
@RestController
@RequestMapping("/api/crops")
@CrossOrigin(origins = "http://localhost:3000")
public class CropController {

    @Autowired
    private CropService cropService;

    @PostMapping("/add")
    public ResponseEntity<Crop> addCrop(
            @RequestBody Crop crop) {

        Crop savedCrop = cropService.saveCrop(crop);
```

```
        return new ResponseEntity<>(
                savedCrop,
                HttpStatus.CREATED);
    }

    @GetMapping("/all")
    public List<Crop> getAllCrops() {
        return cropService.getAllCrops();
    }
}
```

8.4 Weather Management Module Code

Manages weather information and provides weather details required for agricultural activities.

```
@RestController
@RequestMapping("/api/weather")
@CrossOrigin(origins = "http://localhost:3000")
public class WeatherController {

    @Autowired
    private WeatherService weatherService;

    @GetMapping("/{location}")
    public ResponseEntity<Weather> getWeather(
            @PathVariable String location) {

        Weather weather =
                weatherService.getWeather(location);

        return new ResponseEntity<>(
                weather,
                HttpStatus.OK);
```

```
    }
}
```

8.5 Irrigation Guidance Module Code

Provides suitable irrigation guidance based on crop and agricultural requirements.

```
@RestController
@RequestMapping("/api/irrigation")
@CrossOrigin(origins = "http://localhost:3000")
public class IrrigationController {

    @Autowired
    private IrrigationService irrigationService;

    @GetMapping("/guidance")
    public ResponseEntity<Irrigation> getGuidance(
            @RequestParam String crop,
            @RequestParam String soilType) {

        Irrigation guidance =
                irrigationService.getGuidance(
                        crop, soilType);

        return new ResponseEntity<>(
                guidance,
                HttpStatus.OK);
    }
}
```

8.6 Market Price and Government Scheme Module Code

Provides agricultural market price information and government scheme details to farmers.

```
@RestController
@RequestMapping("/api/agriculture")
@CrossOrigin(origins = "http://localhost:3000")
```

```
public class AgricultureController {

    @Autowired
    private MarketPriceService marketPriceService;

    @Autowired
    private SchemeService schemeService;

    @GetMapping("/market-prices")
    public List<MarketPrice> getMarketPrices() {
        return marketPriceService.getAllPrices();
    }

    @GetMapping("/schemes")
    public List<GovernmentScheme> getSchemes() {
        return schemeService.getAllSchemes();
    }
}
```

8.7 Admin Module Code

Provides administrative operations such as managing users, crops, irrigation guidance, market prices, and government schemes.

```
@RestController
@RequestMapping("/api/admin")
@CrossOrigin(origins = "http://localhost:3000")
public class AdminController {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private CropRepository cropRepository;
```

```
@Autowired
private MarketPriceRepository marketPriceRepository;

@Autowired
private SchemeRepository schemeRepository;

@GetMapping("/users")
public List<User> getUsers() {
   return userRepository.findAll();
}

@GetMapping("/crops")
public List<Crop> getCrops() {
   return cropRepository.findAll();
}

@GetMapping("/market-prices")
public List<MarketPrice> getMarketPrices() {
   return marketPriceRepository.findAll();
}

@GetMapping("/schemes")
public List<GovernmentScheme> getSchemes() {
   return schemeRepository.findAll();
}

@DeleteMapping("/crop/{id}")
public String deleteCrop(
       @PathVariable Long id) {
```

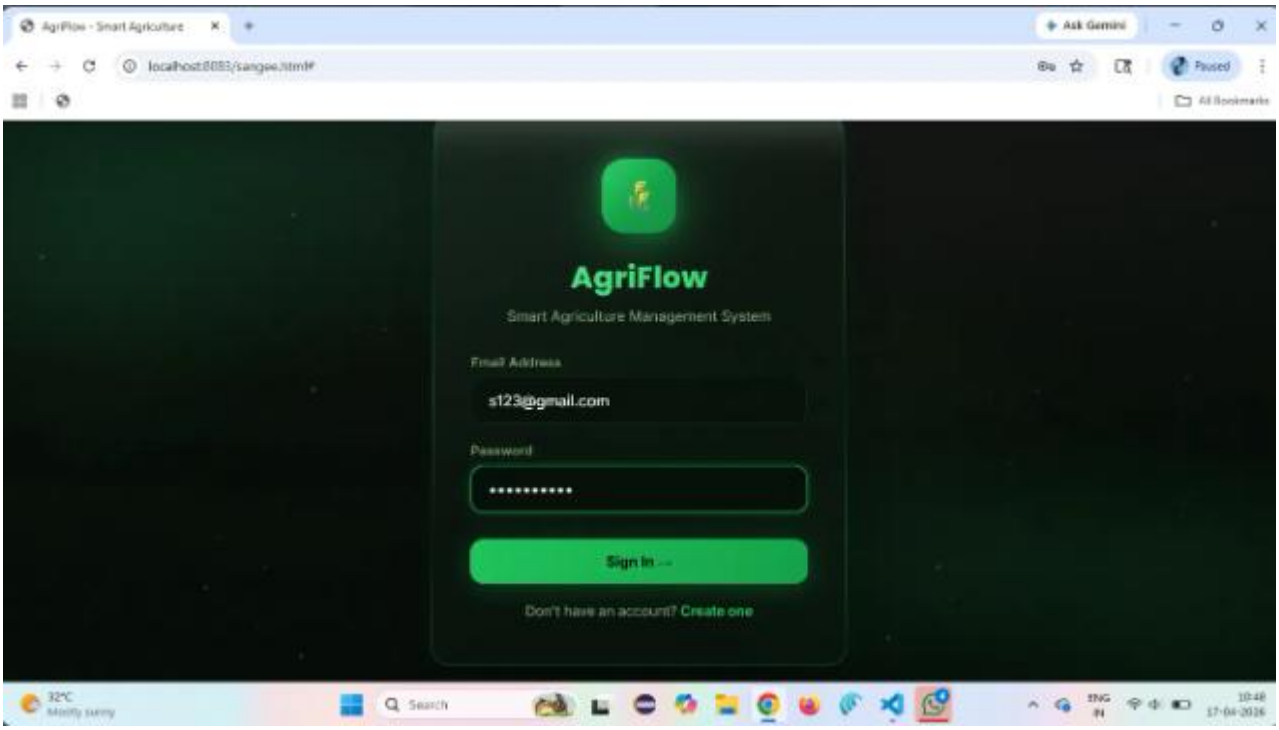
```
        cropRepository.deleteById(id);

        return "Crop Deleted Successfully";
    }
}
```

9.SAMPLE OUTPUT

9.1 Home Page

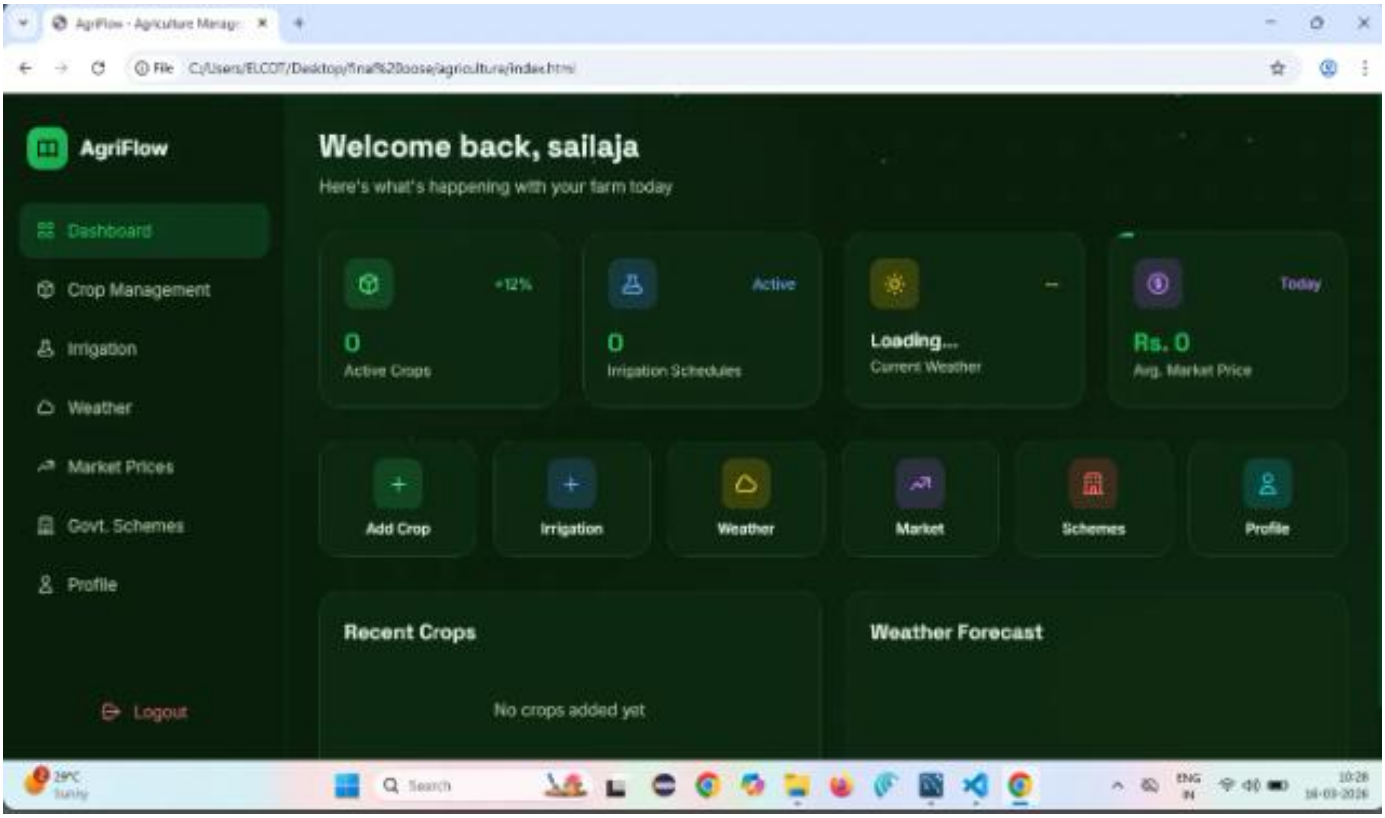
Displays the main interface of the Agriculture Management System.



9.2 Farmer Dashboard

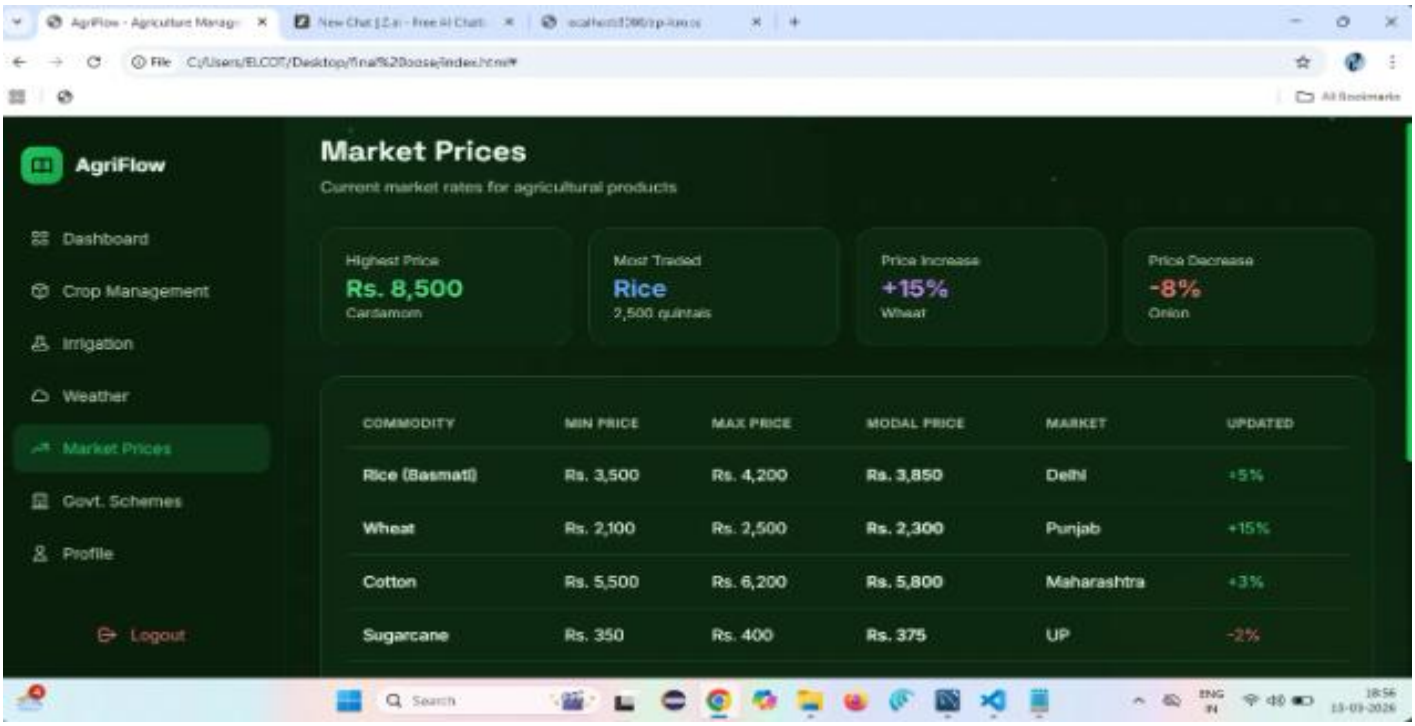
Displays farmer activities and essential agricultural information. Farmers can view crop-related information, check current weather conditions, access irrigation guidance, view agricultural market prices, explore government schemes, and manage their profile. The dashboard provides a simple and user-friendly interface that allows farmers to access important agricultural services from a centralized platform.

.



9.3 Market Prices

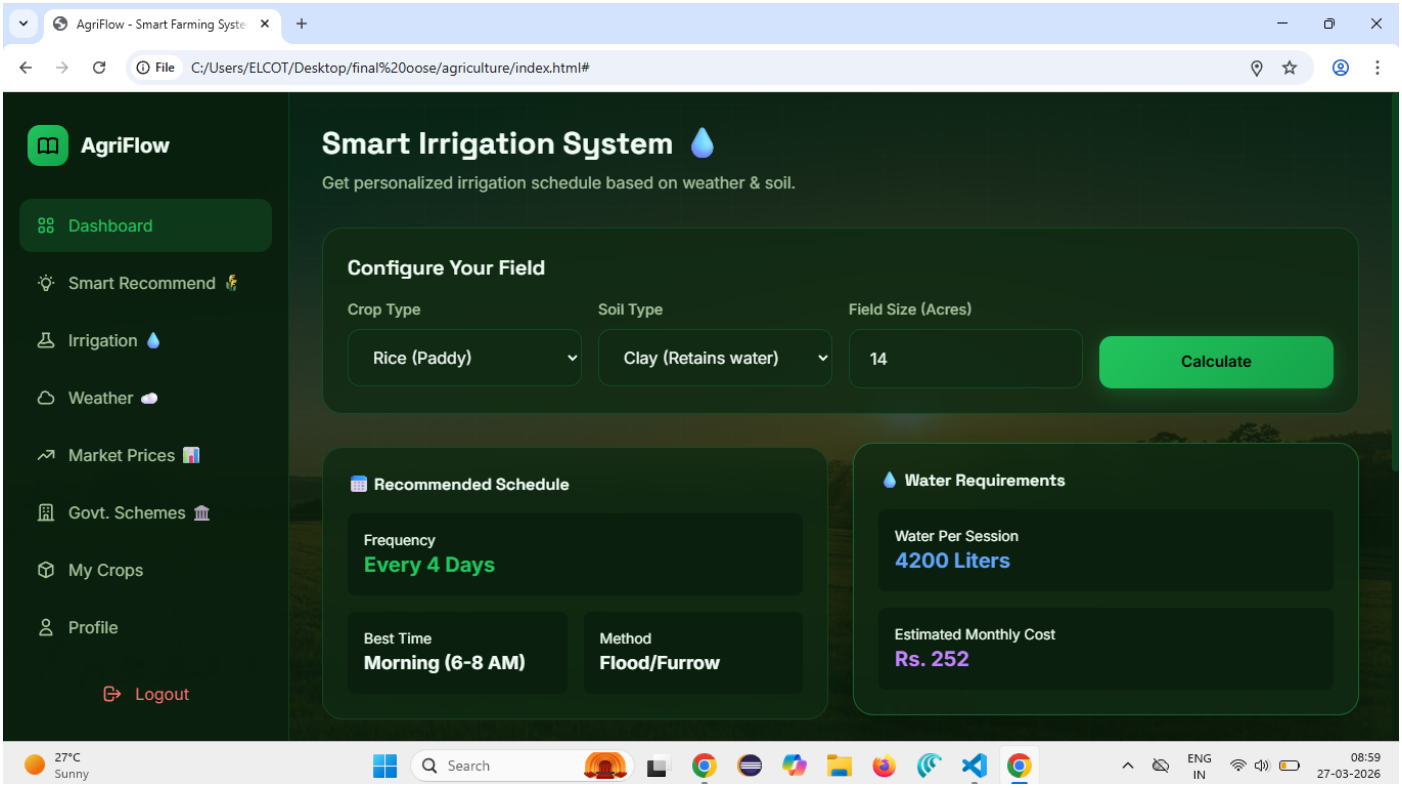
Displays the latest available agricultural market price information for different crops and commodities. Farmers can select the required crop and view its corresponding market price details. This module helps farmers understand current market conditions, compare available prices, and make better decisions regarding the selling of their agricultural products.



9.4 Smart Irrigation System

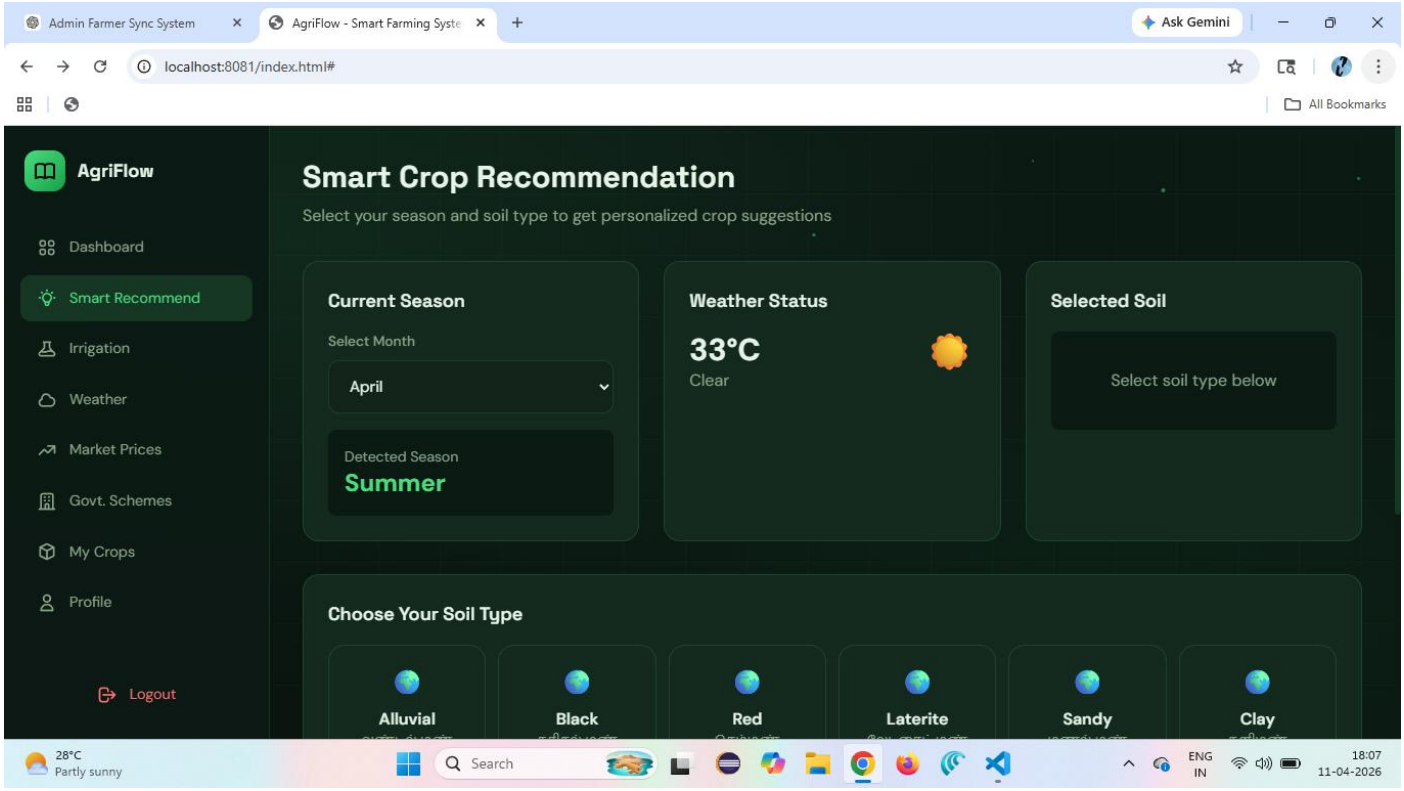
Provides suitable irrigation guidance to farmers based on crop requirements and relevant agricultural conditions. Farmers can select the crop and soil type to receive appropriate irrigation recommendations, including suitable irrigation methods, timing, and frequency. This module helps farmers use water efficiently, avoid over-irrigation or under-irrigation, and maintain healthy crop growth.

.



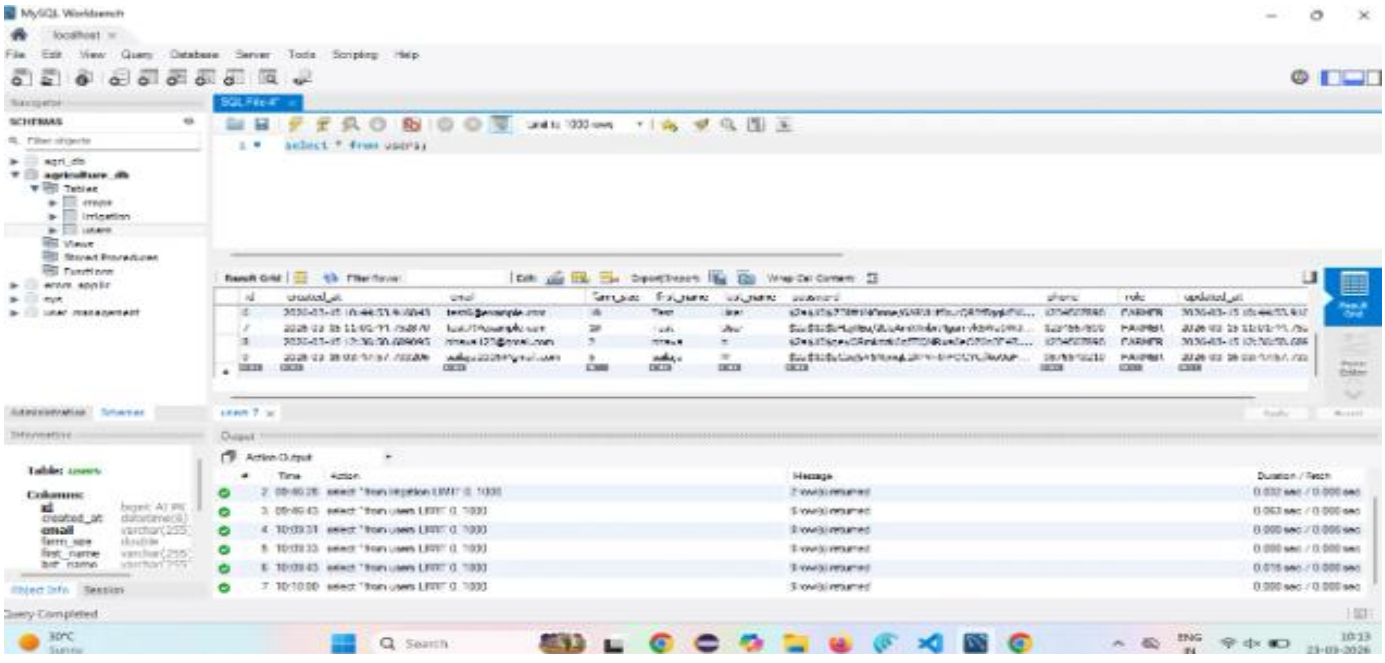
9.5 Crop Recommendation

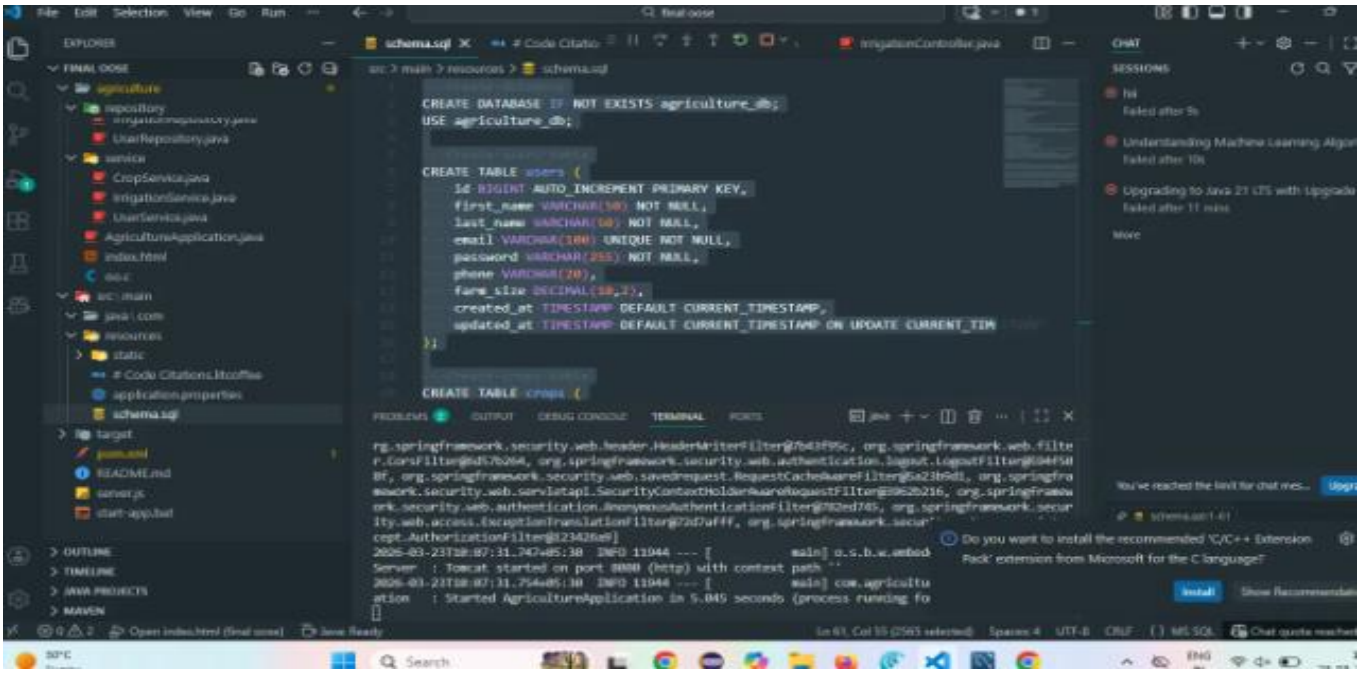
Provides suitable crop recommendations based on important agricultural factors such as **soil type, season, location, and crop requirements**. Farmers can enter the required details and receive suitable crop suggestions through the system. This module helps farmers select appropriate crops, improve cultivation planning, and support better agricultural decision-making.



9.7 Database and Code Overview

Shows the underlying MySQL database storing registered users and their farm-related details, and the backend project structure used during development.





10. ANALYSIS

Testing of the **Agricultural Management System** was carried out module by module to validate both functional correctness and overall system behavior. Functional analysis confirmed that user registration, login, crop management, weather information, irrigation guidance, market price viewing, government scheme access, and administrative operations performed as expected under normal and edge-case inputs. Various test conditions such as duplicate email registration, invalid login credentials, empty fields, unavailable information, incorrect data entries, and invalid requests were also considered to ensure reliable system behavior.

Performance observations showed that the REST API response times remained fast for typical operations due to the lightweight nature of the **Spring Boot** embedded server and the current sample data setup. The system was able to retrieve and display agricultural information efficiently during testing. The application architecture also provides scope for handling larger datasets and more users when deployed with a complete and properly optimized **MySQL database**.

Usability analysis indicated that the responsive **Bootstrap-based interface** adapted well across different desktop and mobile screen sizes. The farmer dashboard provides easy access to crop information, weather details, irrigation guidance, market prices, and government schemes. The administrative dashboard also simplifies the process of managing agricultural information compared to traditional methods where farmers and administrators may need to depend on multiple sources.

Comparative analysis against the existing manual agricultural information system highlighted clear improvements. Accessing agricultural information was reduced from a time-consuming manual process to a few guided interactions. Information management was improved through centralized storage, and administrators gained better control over crop details, weather information, irrigation guidance, market prices, and government scheme details. The system also reduces dependency on scattered information sources and makes important agricultural information easier to access.

The analysis also showed that the centralized approach helps improve the consistency and availability of agricultural information. Farmers can access the required information through a single platform instead of searching through different sources. This can help reduce the time required for information gathering and support more informed decisions related to crop planning, irrigation, market opportunities, and available government schemes.

Overall, the analysis confirms that the **Agricultural Management System** meets its objectives of reducing manual effort, improving access to agricultural information, supporting better decision-making, minimizing information management difficulties, and providing a convenient and efficient digital platform for both farmers and administrators. The system also provides a strong foundation for future enhancements and integration with advanced agricultural technologies.

11. CONCLUSION

The **Agricultural Management System** successfully demonstrates how full-stack web technologies can be applied to modernize and simplify agricultural information management. By replacing traditional and scattered methods of accessing agricultural information with a

centralized digital platform, the system reduces the difficulty of information gathering, improves accessibility, and supports better decision-making for farmers.

The project's layered architecture, consisting of a responsive **Bootstrap-based frontend** and a **Spring Boot backend**, provides a clear separation of concerns and enables smooth communication through REST APIs. Core modules including user registration, authentication, crop management, weather management, irrigation guidance, market price management, government scheme management, and administrative management were designed and tested successfully.

The system provides farmers with an easy way to access **crop information, weather conditions, irrigation guidance, agricultural market prices, and government scheme details** through a single platform. At the same time, administrators can efficiently manage and update agricultural information through the centralized system, ensuring that relevant information can be maintained in an organized manner.

The centralized approach reduces dependency on multiple manual information sources and helps farmers save time while accessing essential agricultural information. The system also supports better utilization of agricultural resources by providing suitable guidance related to crops and irrigation and by making market and government scheme information easily accessible.

This project also provided valuable practical experience in **full-stack web development, REST API design, Spring Boot, MySQL database interaction, responsive UI development, layered architecture, and software testing**. The developed system can serve as a foundation for a more advanced smart agriculture platform in the future, with potential enhancements such as AI-based crop disease detection, IoT-based soil monitoring, real-time market updates, advanced weather forecasting, and mobile application support.

12. FUTURE WORK

The **Agricultural Management System** can be further enhanced with advanced technologies and additional features to improve its functionality, scalability, and usefulness for farmers.

• Integration with a production-grade **MySQL/PostgreSQL database** for secure, reliable, and scalable storage of agricultural information.

• Integration with **real-time weather APIs** to provide accurate and updated weather conditions and forecasts for different locations.

• Development of a dedicated **Android and iOS mobile application** to provide convenient access to agricultural services for farmers.

• Implementation of **AI-based crop recommendation** to suggest suitable crops based on soil type, season, location, and other agricultural conditions.

• Integration of **IoT-based soil sensors** to monitor soil moisture, temperature, and other important soil parameters in real time.

• Implementation of **AI-based plant disease detection** to identify crop diseases from uploaded plant images and provide suitable guidance.

• Integration of real-time **agricultural market price APIs** to provide updated commodity prices from different markets.

• Implementation of automated **SMS, email, and application notifications** for important weather alerts, market price updates, irrigation guidance, and government scheme information.

• Cloud deployment using **AWS, Microsoft Azure, or Google Cloud Platform** for improved scalability, availability, and accessibility.

• Development of advanced **analytics and dashboards** to help administrators analyze crop information, market trends, user activities, and agricultural data.

• Implementation of **location-based agricultural services** using GPS to provide farmers with relevant information based on their geographical location.

• Implementation of **JWT-based authentication and role-based access control** for enhanced system security and protection of user information.

13. REFERENCES

1. W3Schools – HTML, CSS, JavaScript, Bootstrap, and web development learning resources.
2. GeeksforGeeks – Java, Spring Boot, REST API, database programming, and software development concepts.

3. Spring Boot Official Documentation – Backend development, REST API implementation, and Spring Boot framework concepts.
4. Oracle Java Documentation – Java programming language features, APIs, and standard libraries.
5. Bootstrap Documentation – Responsive user interface design and frontend components.
6. Spring Data JPA Reference Documentation – Repository layer implementation and database interaction.
7. MySQL Documentation – Relational database management, SQL concepts, and database operations.
8. MDN Web Docs – HTML, CSS, JavaScript, and modern web development concepts.
9. Open-Meteo Documentation – Weather API and weather data integration for web applications.
10. Spring Framework Documentation – Dependency injection, REST controllers, and application development concepts.

Fantasy Solution

Find ur Passion...

Date: 28.05.2026

Sangeetha M,
Reg.No: 810023205105,
Department of Information Technology,
University College Of Engineering,
Bharathidasan Institute of Technology (BIT) Campus,
Tiruchirappalli - 620 024.

Dear Sangeetha,

Sub: Internship

On behalf of **FANTASY SOLUTION**. We would like to inform you that you have been accepted as one of our interns. We are pleased to inform you that you have been accepted as per the company requirements for the internship with **FULL STACK WEB DEVELOPMENT**.

Your internship wil l begin from **01st June 2026 to 30th June 2026**. You would your performance would be assessed and appraised.

FANTASY SOLUTION TRICHY

Authorized Signature
(With Seal)

#16, Samnath Plazza, 3rd Floor, Melapudur, Trichy - 620 001.
Ph : 0431-4250535 | Mob : 90430 95535
www.fantasysolution.in | info@fantasysolution.in



FANTASY SOLUTION

Fine Us Passion...

SOFTWARE DEVELOPMENT AND TRAINING

Certificate of Accomplishment

This is certify that Mr./Miss. SANGEETHA M (810023205105)

Student of UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS) - TRICHY

underwent Inplant Training / Internship / Course on FULL STACK DEVELOPMENT (JAVA)

concern from 01/06/2026 to 30/06/2026 issued on 30/06/2026

Technical Incharge

Manager

CEO

ATTENDANCE CERTIFICATE FOR INDUSTRIAL TRAINING / INTERNSHIP

(Attendance Certificate to be signed by the competent authority of the industry mentioning the period of Industrial Training / Internship)

The Student Mr./ Ms SANGEETHA M (Reg.No.) 810023205105

Studying FULL STACK WEB DEVELOPMENT programme at UNIVERSITY COLLEGE OF ENGINEERING BIT CAMPUS ANNA UNIVERSITY, TIRUCHIRAPPALLI College in semester 7th at Department of INFORMATION TECHNOLOGY has attended Industrial Training / Internship from 01/06/2026 to 30/06/2026. It is certified that he / she has completed / not completed the Industrial Training / Internship in our FANTASY SOLUTION organization / institution.

Signature

Name and Designation of the Officer
Seal of the Organization